

FRESH & QUICK

USER MANUAL

Technical Reference & User Guide for Customer, Vendor, Delivery Partner & Admin Portals

Version 2.0 — updated for the Figma-kit redesign
September 2026

Table of Contents

Chapter 1 — Introduction & System Overview.....3

Chapter 2 — Getting Started.....5

Chapter 3 — Customer Module.....7

Chapter 4 — Vendor Module.....16

Chapter 5 — Delivery Partner Module.....22

Chapter 6 — Admin Module.....24

Chapter 7 — Cross-Cutting Features.....28

Appendix A — Technical Architecture & Stack.....31

Appendix B — Database Schema Reference.....33

Appendix C — Seeded Test Accounts.....35

Appendix D — Glossary of Terms.....39

Chapter 1 — Introduction & System Overview

Fresh & Quick is a full-stack, multi-portal food and local-goods delivery platform, built as a functional, from-scratch clone of the delivery-app experience popularised by apps like DoorDash and Uber Eats. It is not a single application but four cooperating web portals that share one Next.js codebase and one Postgres database, each portal gated to its own role so that a login on one portal never grants access to another.

The platform's marketplace has grown beyond restaurants: as of this manual, 77 seeded stores span 11 categories — Restaurants plus Grocery, Convenience, Alcohol, Health, Retail, Pet, Flowers, Baby, Personal Care and Electronics — all browsable, orderable and trackable through the same customer experience.

1.1 The Four Portals

- Customer App ("/customer") — anonymous browsing of every store and category, cart management, checkout with contact details and a choice of payment method, and order tracking.
- Vendor Portal ("/vendor") — store owners (restaurants and every other marketplace category) manage their menu/catalog, open or close their store, set delivery fee and promo text, and work incoming orders through a kanban board.
- Delivery Partner Portal ("/delivery") — delivery partners go online, claim ready orders, and advance them from picked-up to delivered while sharing a live location ping.
- Admin Portal ("/admin") — platform staff see every order, restaurant and delivery partner, suspend a misbehaving store, and reassign a stuck delivery to a different partner.

Each portal is reachable at its own top-level path and has its own dedicated sign-in page; the platform enforces this separation both in the UI (separate login forms, separate navigation shells) and at the data layer (Postgres row-level security policies scoped to each role, plus service-role-only write paths for anything security-sensitive).

1.2 How the System Fits Together

All four portals are pages within one Next.js application. Every data-changing action goes through a Next.js API route running on the server, which uses a privileged "service role" Supabase key to talk to Postgres — this key never reaches the browser. Reads that are safe for a signed-in user's own browser to perform directly (e.g. browsing restaurants, viewing your own orders) go through Supabase's auto-generated REST layer (PostgREST) protected by row-level security policies, so that even a technically savvy customer cannot read or write another customer's data by crafting their own request.

A separate automation layer, n8n, listens for events such as an order being placed and drives cross-portal side effects — resolving a mock payment, notifying the vendor, and so on — by calling a small set of internal API routes ("/api/internal/*") protected by a shared secret header. If n8n is not running, the same checkout still works: the checkout route falls back to resolving payment itself after a short wait, so the demo experience never depends on n8n being present.

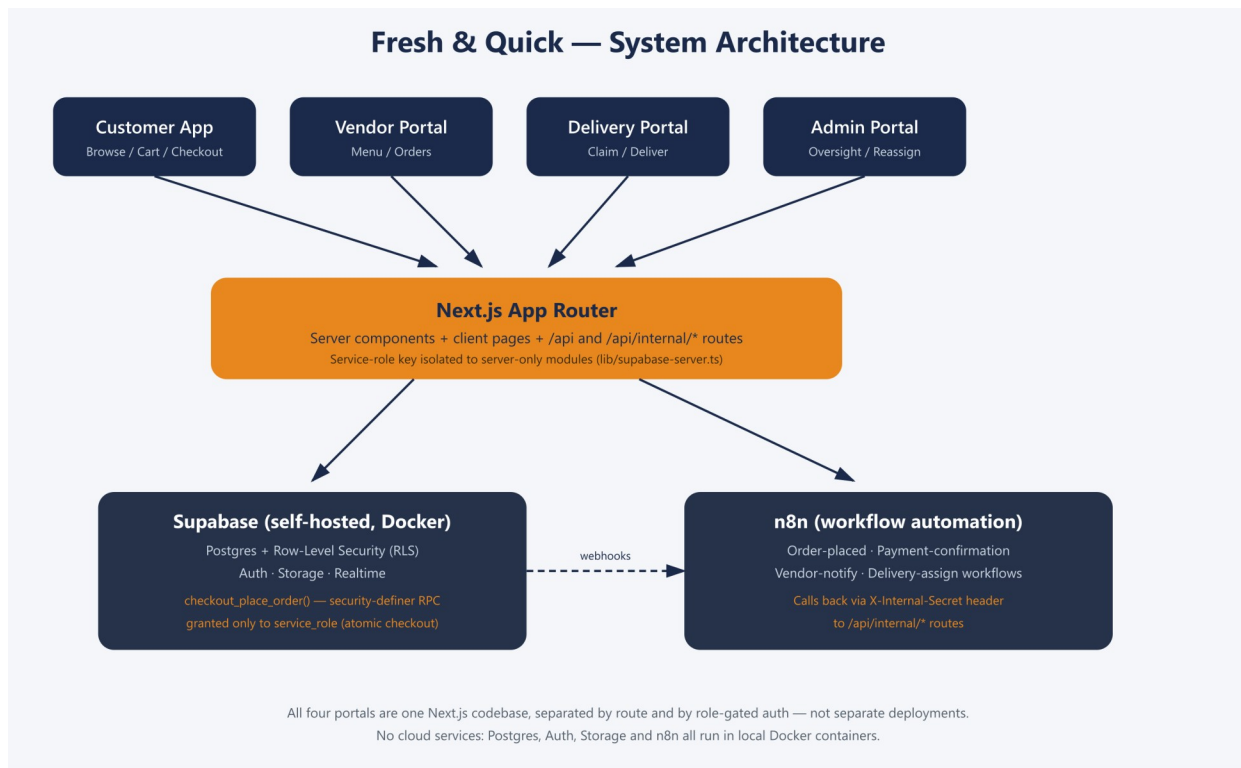


Figure 1.1 — System architecture: four portals, one Next.js codebase, Supabase Postgres with row-level security, and an n8n automation layer reachable via internal webhooks.

1.3 Who This Manual Is For

This manual is written for two audiences at once: (1) anyone actually using the platform — a customer placing orders, a restaurant or store owner managing their menu, a delivery partner fulfilling orders, or platform staff running the admin console — and (2) a technical reader who wants to understand how the system is built, summarised in the Technical Appendix at the end of this manual.

1.4 What's New in Version 2.0

This edition replaces every screenshot with captures of the current build and adds the changes made since version 1.0:

- New look: an orange / navy / green palette with pill-shaped buttons, rounded cards and colored section backgrounds on all four portals. Colors used behind white text were darkened so buttons, badges and dashboard tiles are easy to read.
- Always signed out after a restart: when the application server is restarted, every browser starts signed out and you must sign in again. Normal page navigation and reloads keep you signed in while the server keeps running.
- My Profile: every portal now shows your name with a Reset password link — in the account menu for customers and in the left sidebar for vendors, delivery partners and admins.
- Your chosen delivery location is remembered between visits; checkout name, email and address always start blank.
- Delivery partner and admin portals now use the same left-sidebar layout as the vendor portal.

Chapter 2 — Getting Started

2.1 Local Environment Prerequisites

Fresh & Quick runs entirely on infrastructure you control — there is no cloud/hosted Supabase project and no external database. To run it locally you need:

- Node.js 20 or newer
- Docker Desktop, running, before you start the stack (Supabase's local stack and the optional n8n container both run as Docker containers)

2.2 Starting the Application

1. `npm install` — installs the Next.js application's dependencies.
2. `npx supabase start` — starts local Postgres, Auth, Storage and Realtime in Docker containers.
3. Copy the API URL and anon key that step 2 prints into `.env.local` (see `.env.example` for the exact variable names).
4. `npm run app:seed` — applies every database migration and loads the seed data (77 stores, 80 test accounts, menu items, and cuisine taxonomy).
5. `npm run dev` — starts the Next.js application at `http://localhost:3000`, which redirects to the customer app by default.

For a production-style build and start (matching how the app is actually deployed) rather than the hot-reloading dev server, use `npm run app:build` followed by `npm run app:start`; both are also available as `npm run app:stop` to shut everything down together.

Signing in after a restart: because the app treats every server start as a fresh session, you will be signed out of every portal the first time you open it after running `npm run app:start` (or restarting `npm run dev`). Simply sign in again.

2.3 URLs at a Glance

Portal	URL	Notes
Customer app	<code>http://localhost:3000/customer</code>	No login needed to browse
Vendor portal	<code>http://localhost:3000/vendor/login</code>	Store owners
Delivery portal	<code>http://localhost:3000/delivery/login</code>	Delivery partners
Admin portal	<code>http://localhost:3000/admin/login</code>	Platform staff, no self-signup
Supabase Studio	<code>http://localhost:54323</code>	Database admin UI

2.4 Test Accounts

The seed data includes one demo account per role, ready to sign in with immediately — see Appendix C for the complete account list, including every seeded restaurant and non-restaurant store's own vendor login. New customer, vendor and delivery-partner accounts can also be created at any time from each portal's own "Sign up" toggle; admin accounts are seeded only, since the platform has no admin self-registration flow by design.

2.5 Testing Multiple Roles at Once (Multi-Port Mode)

By default the app runs entirely on `http://localhost:3000`, with each portal distinguished only by its login path (`/customer`, `/vendor/login`, `/delivery/login`, `/admin/login`). Because all four portals share that one origin, and a browser's sign-in session is stored per-origin, logging into a second portal in another tab on the same port silently replaces the session of whichever role was signed in first — fine for testing one role at a time, but not for having a customer, vendor, delivery partner, and admin all signed in simultaneously to watch an order move through the whole system.

For that, start the app in multi-port mode instead: `npm run app:start:all-roles` (or `.\scripts\start.ps1 -Dev -AllRoles`) runs the same application four times, one per port, so each portal is its own browser origin with its own independent session:

Role	URL (multi-port mode)
Customer	<code>http://localhost:3000/customer</code>
Vendor	<code>http://localhost:3001/vendor/login</code>
Delivery Partner	<code>http://localhost:3002/delivery/login</code>
Admin	<code>http://localhost:3003/admin/login</code>

All four can be open in tabs of the same ordinary browser window at once, with no Incognito windows or separate browser profiles needed. Use `npm run app:start:dev` when you only need one role at a time; use `app:start:all-roles` when you want to watch an order cross from customer to vendor to delivery to admin live. `npm run app:stop` (or `app:stop:all-roles`) shuts down whichever mode is running.

Chapter 3 — Customer Module

The customer app is the storefront: browsing, cart, checkout, and order tracking. Browsing every store and category requires no account at all; an account is required only once you try to check out, so that guest browsing is always frictionless.

3.1 Home Page

The home page is a DoorDash-style layout with a persistent left sidebar, a header with an address picker and a Delivery/Pickup toggle (Pickup is a disabled placeholder — no pickup flow exists in this version), and a body that mixes restaurants with every other store category into cuisine- and category-grouped horizontal carousels.

- Left sidebar — one-tap jumps to Home, Restaurants, and each of the ten non-restaurant categories (Grocery, Convenience, Alcohol, Health, Retail, Pet, Flowers, Baby, Personal Care, Electronics), plus Orders and Account.
- Header search box — finds restaurants or stores by name or cuisine/category label.
- Quick filters — Rating, Delivery fee, and "Under 30 min", plus a sort dropdown (Distance, Rating, Delivery fee, Prep time).
- Cuisine category chips — one-tap filtering by cuisine (Bakery, Chinese, Indian, Italian, and more).
- Store carousels — sections such as "Popular near you" and "Quick delivery" mix restaurants with grocery, pet, beauty and every other category by default; each carousel shows left/right scroll arrows once there is more content than fits on screen.

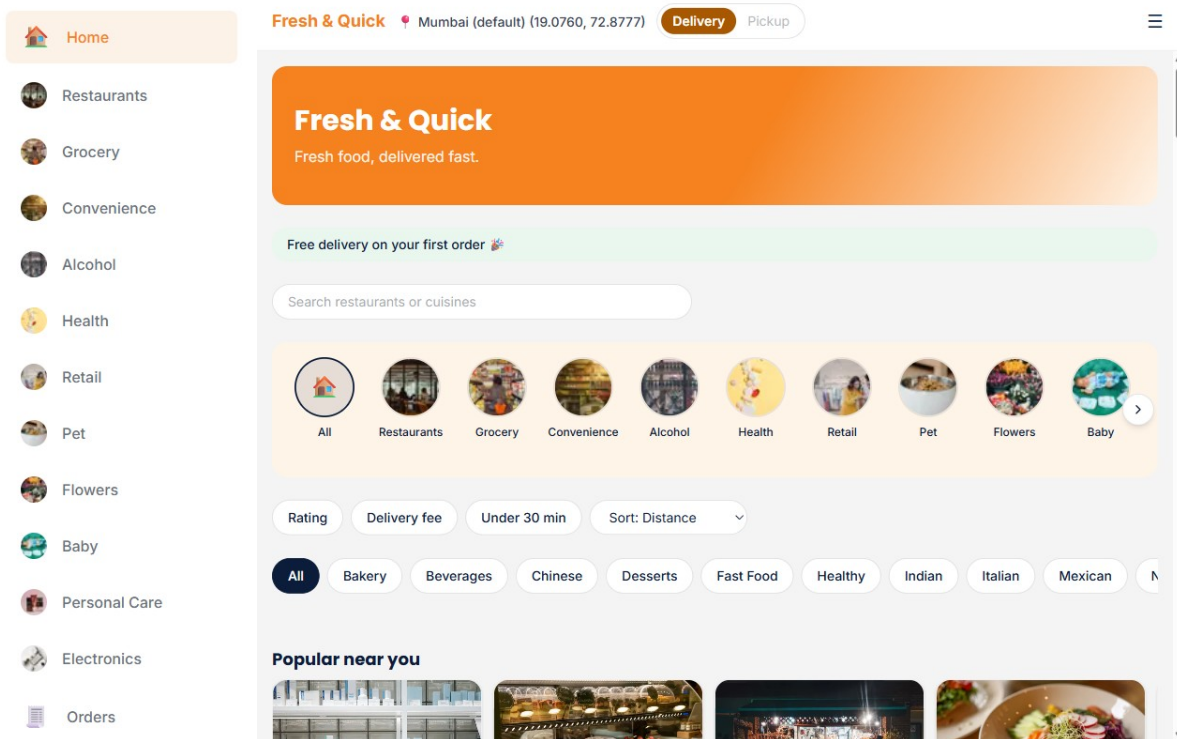


Figure 3.1 — Customer home page: sidebar categories, header search and delivery toggle, and a mixed feed of restaurants and other stores.

Step by step: finding something to order

1. Open <http://localhost:3000/customer> — it lands on the home page.
2. Either tap a category in the left sidebar, tap a cuisine chip, type into the search box, or simply scroll the curated carousels.
3. Tap any store card to open that store's full menu or catalog page.

3.2 Restaurant & Store Pages

Every store — whether a restaurant or a grocery, pet, electronics, or any other category store — uses the same browse-and-add layout: a banner, the store's name, cuisine/category tags, rating and estimated prep time, a menu search box, a real-rating summary block, a horizontal “Featured items” carousel (the first available items with a photo), category filter pills, and the full item catalog laid out as a 4-column grid of cards (2 columns on narrow screens), grouped by category. Each item card shows its photo, name, description, price, a veg/non-veg indicator dot for food items, and a circular “+” button that adds one unit to the cart.

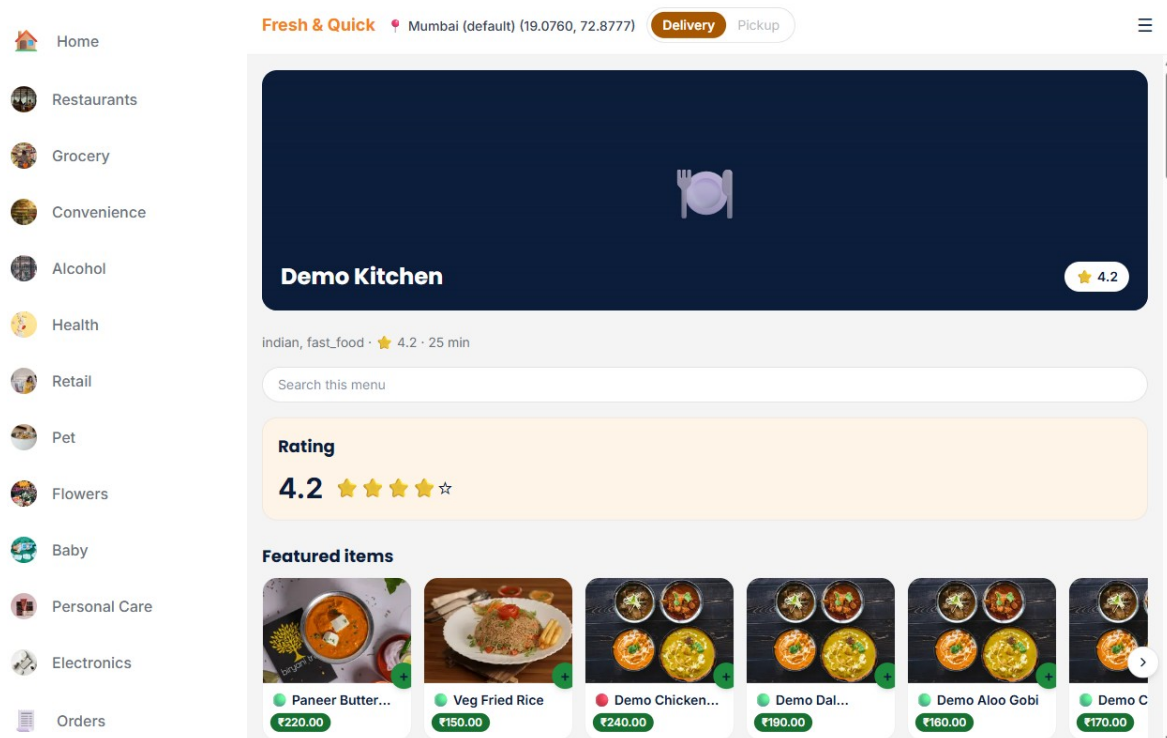


Figure 3.2 — Demo Kitchen's store page: the store rating, a Featured items carousel, category pills, and the item catalog as a card grid, with the persistent cart panel on the right once an item has been added.

Ten non-restaurant categories use the identical catalog layout, so everything explained above for a restaurant menu applies equally to a grocery, pet-supply, or electronics store's catalog page.

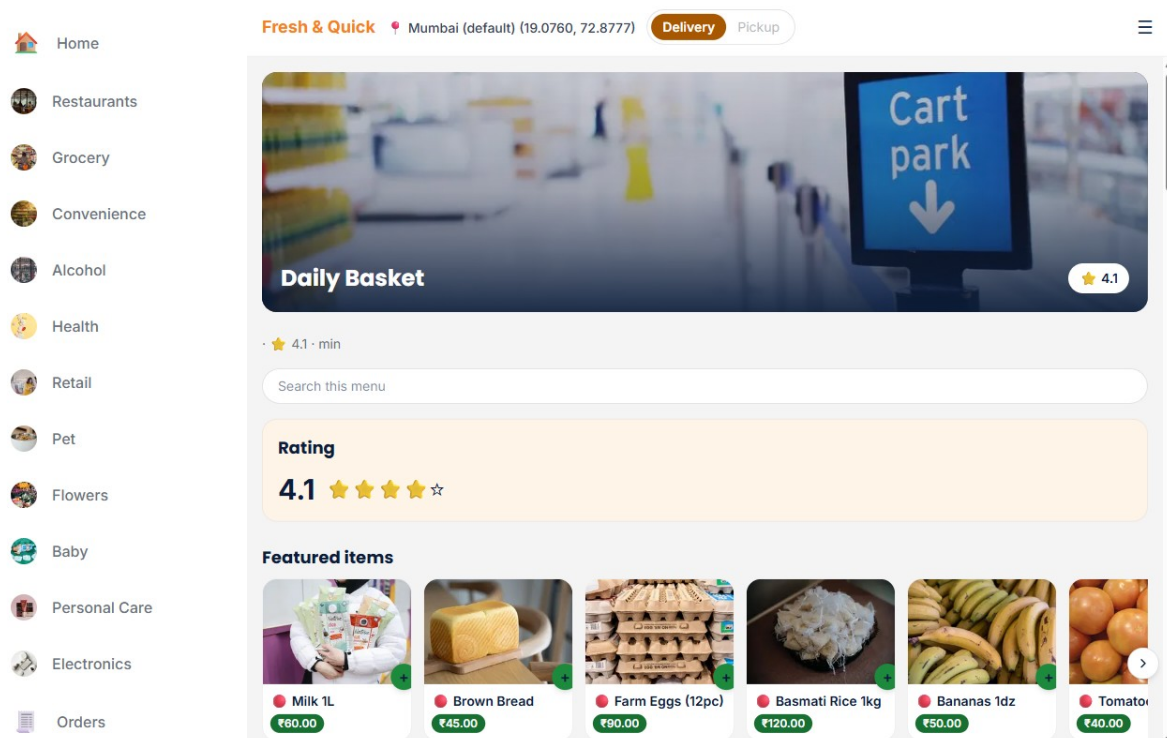


Figure 3.3 — Daily Basket, a Grocery-category store, browsed with the exact same rating/featured-items/grid layout as a restaurant menu.

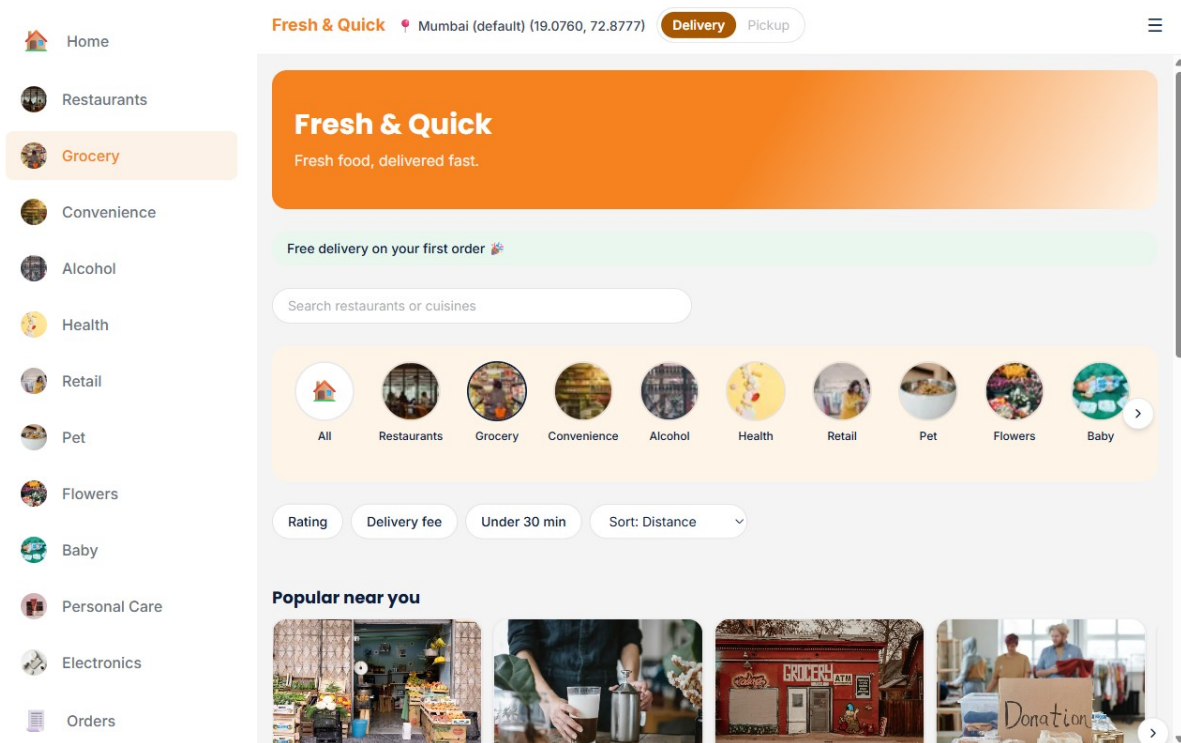


Figure 3.4 — The home page filtered to the Grocery category via the sidebar, showing every seeded grocery store.

3.3 Cart

The cart is single-store only by design: adding an item from a different store than the one already in your cart prompts you to clear the cart first, rather than silently mixing items from two different stores into one order. On a store's own page, the cart appears as a persistent panel docked to the right of the menu, listing every item with quantity +/- controls, a per-item note field, a Remove link, the running subtotal and delivery fee, and a whole-order note field separate from any per-item notes.

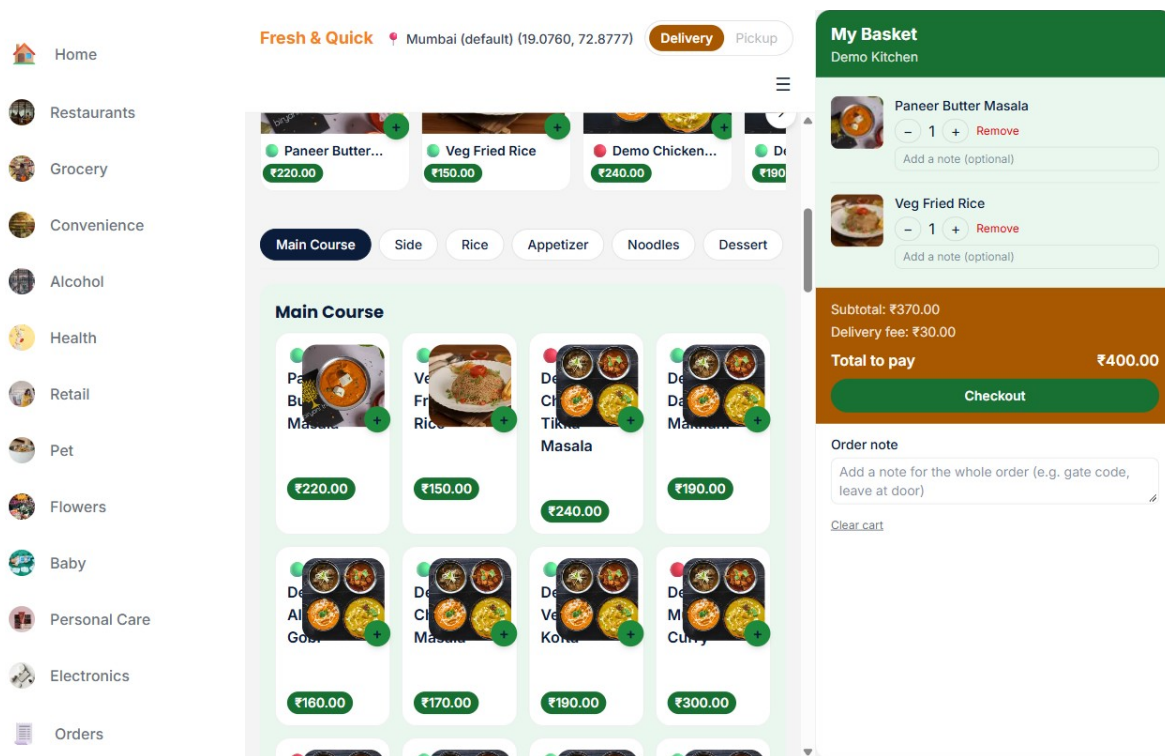


Figure 3.5 — The cart panel with an item added: quantity control, per-item note, subtotal/delivery-fee/total breakdown, and the whole-order note field.

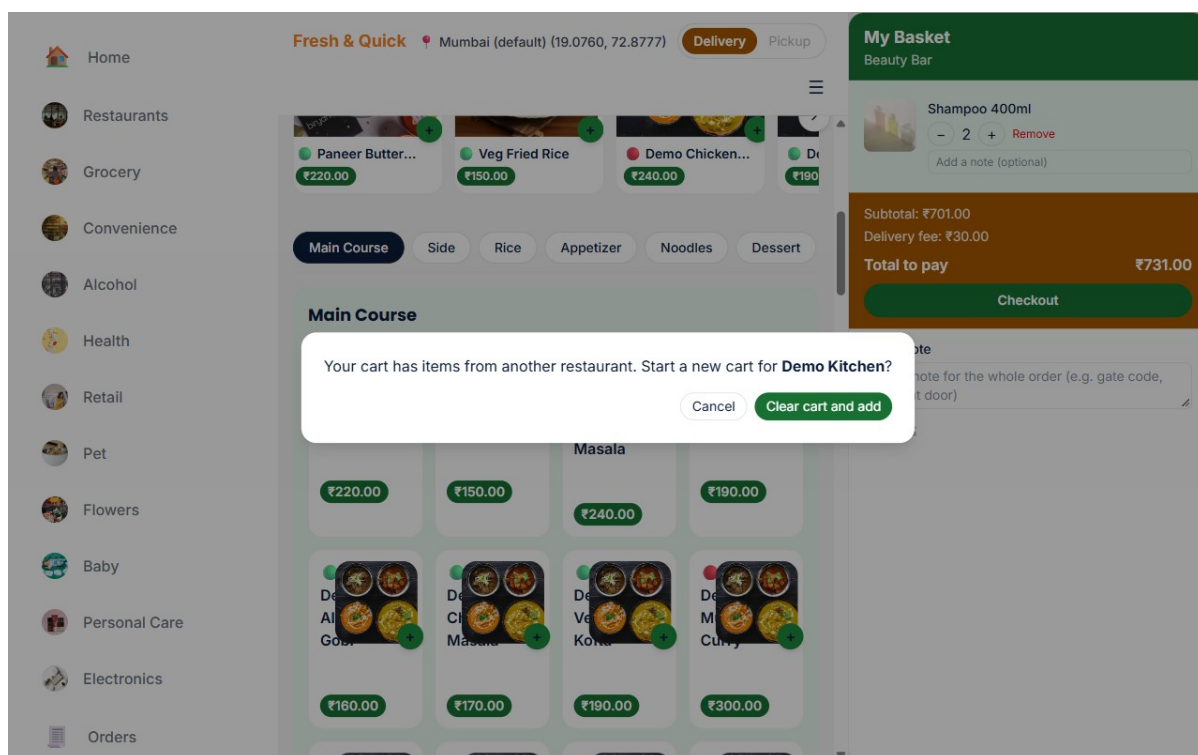


Figure 3.5a — Adding an item from a different restaurant asks whether to start a new cart: a cart can only hold items from one store at a time.

Step by step: building an order

1. On a store's page, tap the + button next to each item you want.

2. Use the – / + controls in the cart panel to adjust quantity, or Remove to drop an item.
3. Optionally add a per-item note (e.g. "no onions") or a whole-order note (e.g. a gate code or "leave at door").
4. Review the subtotal, delivery fee and total, then select Checkout.

3.4 Checkout

Checkout is a single page: contact details, delivery address, and payment method stacked in one column on the left. The persistent cart panel on the right — the same panel visible on every store page — shows the order summary and the single Place Order button for the whole app; checkout no longer shows a second, separate order-summary card. Reaching checkout while signed out redirects you to the login page and returns you to checkout automatically once you sign in.

The checkout page collects three things before it will let you place an order:

- Contact details — a recipient name and email, pre-filled from your account profile where available, used for order-update notifications.
- Delivery address — a real form: Address 1, Address 2 (optional), City, State, and Pincode, all required except Address 2. Saved locally so it's pre-filled on your next order.
- Payment method — Mock Card, Mock UPI, or Cash on Delivery, each with its own required fields.

The Place Order button stays disabled (greyed out) until every required field for the selected payment method validates — this prevents an incomplete or malformed order from ever reaching the server.

Figure 3.6 — Checkout with Mock Card selected: Contact details, the Address 1/2/City/State/Pincode delivery address form, card number/expiry/cardholder-name fields, and the persistent cart panel's order summary and Place Order button on the right.

Figure 3.7 — Checkout with Mock UPI selected, showing the UPI ID field that replaces the card fields; the delivery address form stays the same regardless of payment method.

Payment methods explained

- Mock Card — requires a 16-digit card number, an MM/YY expiry, and a cardholder name. Resolves to success or failure at random, roughly an 80% success rate, to simulate a real payment gateway.
- Mock UPI — requires a UPI ID (e.g. name@bank). Same ~80% success/failure simulation as Mock Card.
- Cash on Delivery — no extra fields required, and always succeeds immediately, since payment happens in person at delivery.

All prices throughout the platform are calculated and stored to the exact paisa (1/100 of a rupee) using integer arithmetic, never floating-point multiplication — this avoids the rounding errors that plain JavaScript float math would otherwise introduce into money calculations.

Placing an order is processed atomically on the server: the order row, its line items, and the payment record are all created together through a single database transaction (a Postgres function reserved for the server's own privileged connection), so a partial failure midway through checkout cannot leave a half-created order behind. If a local n8n automation instance is running, it resolves the mock payment within a few seconds; if not, the checkout route itself waits out a ~10-second timeout and then resolves the payment in-process, so every order completes correctly either way — just faster with n8n connected.

Step by step: completing checkout

1. Fill in or confirm your recipient name and email under Contact details.
2. Choose a payment method and fill in its required fields.
3. Review the order summary — line items, subtotal, delivery fee, and total — on the right.
4. Select Place Order once the button is enabled.
5. Wait for payment to resolve; you are then taken to the order confirmation page.

3.5 Order Confirmation & Tracking

After a successful checkout, the customer lands on that order's confirmation page, which shows a four-step status timeline — Placed, Preparing, On the way, Delivered — instead of a plain text status line. Once a delivery partner is assigned to the order, a bordered coordinate box shows their last reported location in place of a live map (no Google Maps key is configured in this environment, so this is a readable stand-in rather than an interactive map).

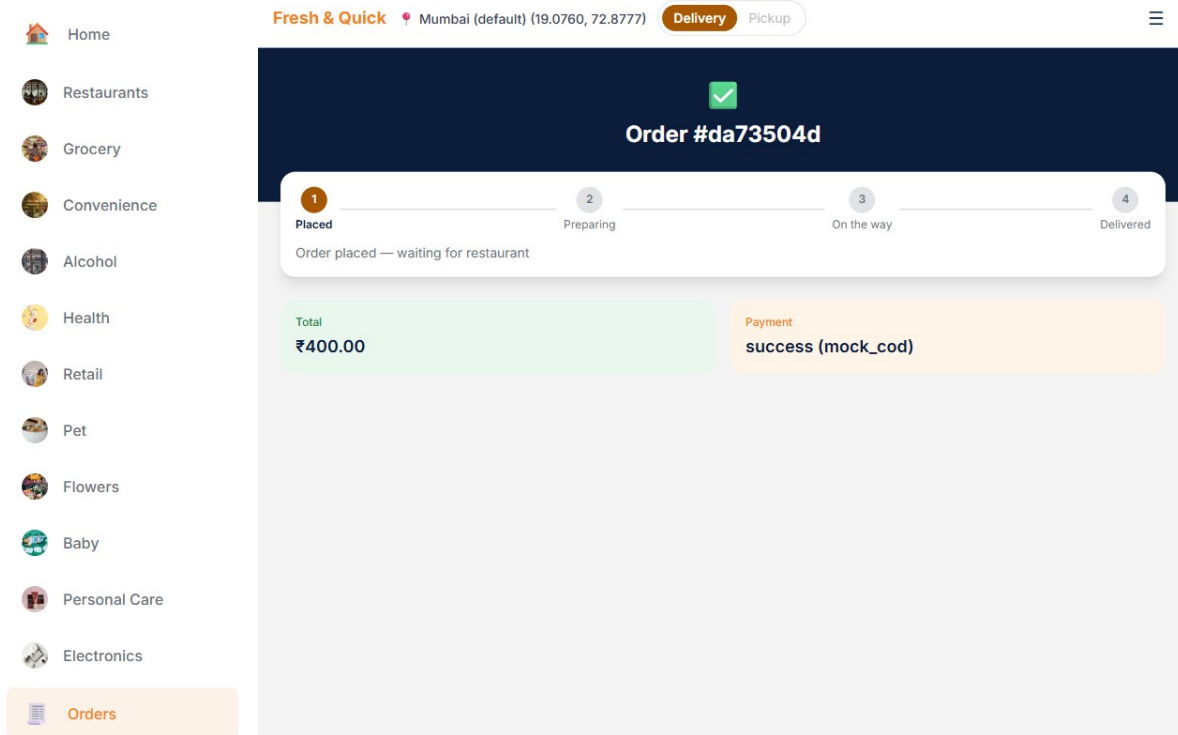


Figure 3.8 — Order confirmation page for a freshly placed Cash on Delivery order, showing the order's line items, totals, and current status.

The confirmation page polls the server periodically for status updates and stops polling once the order reaches a terminal state (Delivered or Cancelled), so it does not poll forever after the order journey has ended.

3.6 Signing In & Creating an Account

The customer sign-in page accepts a registered email and password; a toggle on the same screen switches to account creation for a brand-new customer. Both flows use Supabase's own authentication service under the hood, so passwords are never stored or handled directly by the application code.

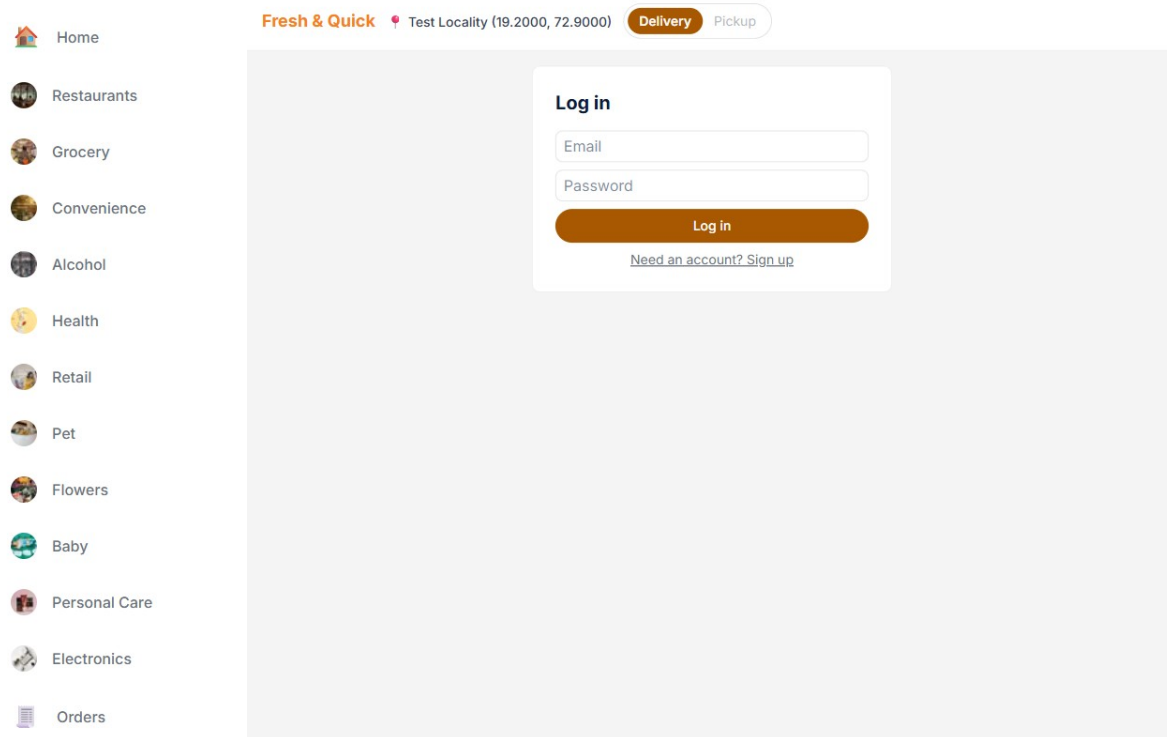


Figure 3.9 — Customer sign-in page. The same screen toggles to "Sign up" for new customers.

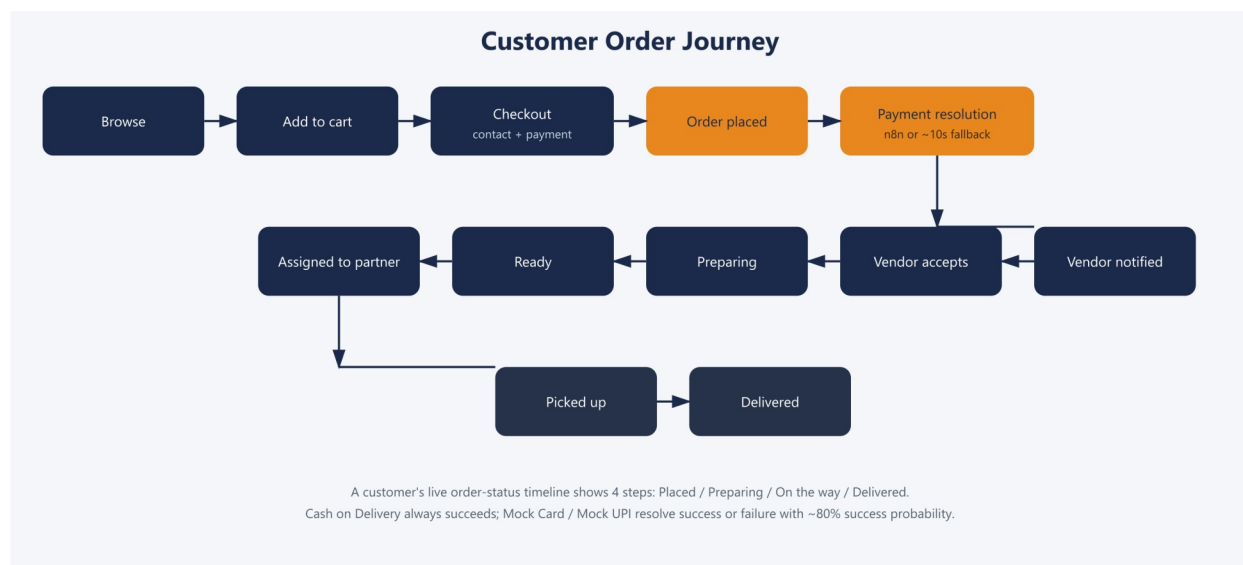


Figure 3.10 — The full customer order journey from browsing to delivery.

3.7 Account Menu

Once signed in, the Sign In / Sign Up buttons in the left sidebar disappear, and the hamburger icon in the top-right of the header opens the account menu: your name, Orders (a full list of your past and current orders — tap one to see its live status), Wallet (the payment methods available at checkout), Help (a short FAQ on how to use the site), and Sign out. Under your name is a Reset password link: click it and a password-reset email is sent to your account address, and the link changes to “Reset email sent”.

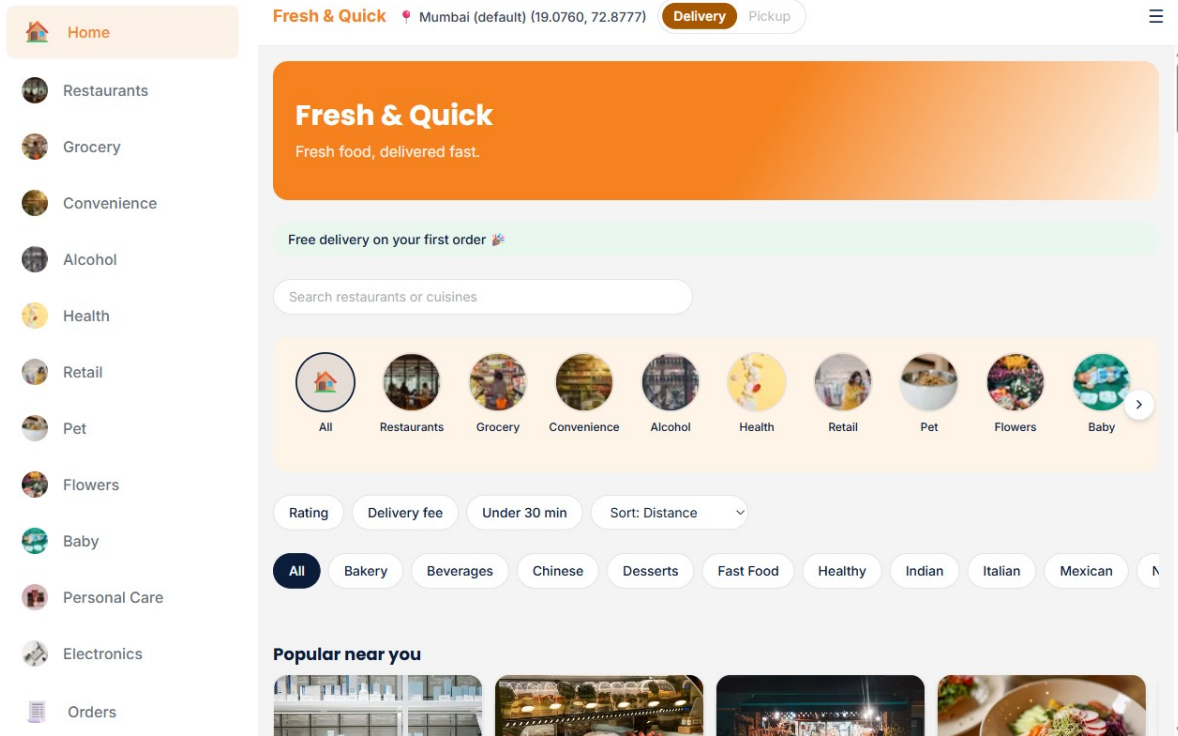


Figure 3.11 — The account menu open from the header hamburger icon, showing the signed-in customer's name with a Reset password link, and the Orders / Wallet / Help / Sign out links.

3.8 Troubleshooting: Unexpected Errors on an Order Page

If an order-tracking page shows an unexpected error instead of your order's status, the most common cause is being signed into a different role's account in another browser tab on the same localhost:3000 origin — for example, signing into the vendor portal in one tab silently replaces the customer session an already-open order-tracking tab was relying on, so that page's next refresh tries to load your order as the vendor instead of as yourself. The fix is simply to sign in again as the customer on the order page: it now detects this automatically and sends you back to the login screen instead of showing a raw error. To avoid this altogether when you want more than one role signed in at once (for example, to watch an order move from customer to vendor to delivery to admin), use the multi-port mode described in section 2.5, which gives each role its own browser origin and its own independent session.

Chapter 4 — Vendor Module

Every seller on the platform — whether a restaurant or any of the ten non-restaurant categories — is a "vendor" account with access to the same vendor portal. A vendor signs in separately from customers, at its own login

page, and manages exactly one store: its menu/catalog, its open/closed status, its delivery settings, and its incoming orders.

4.1 Vendor Sign-In & Sign-Up

The vendor login page is a plain, centered form with no sidebar (the sidebar only appears once signed in). A "New restaurant? Sign up" toggle on the same screen lets a brand-new store register directly: email, password, store name, one or more cuisine/category tags, and a latitude/longitude stub standing in for a real address (no mapping API key is configured in this environment).

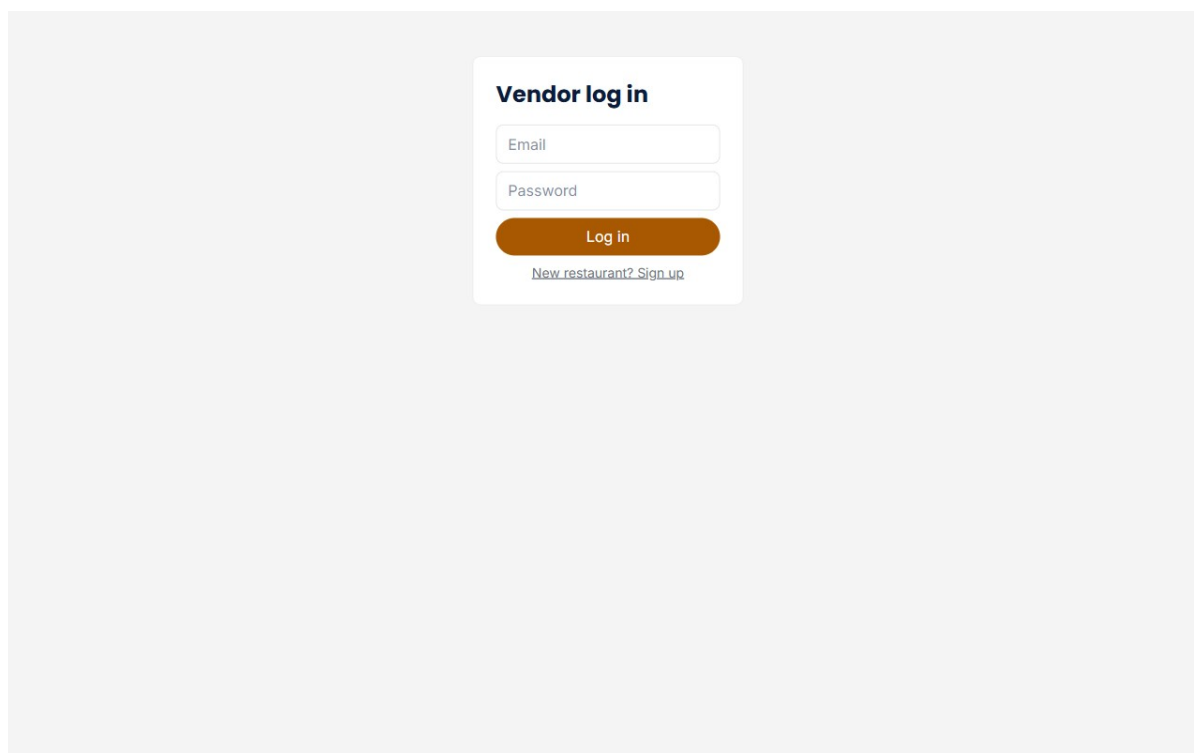


Figure 4.1 — Vendor sign-in page (no sidebar); the same screen toggles to new-restaurant sign-up.

A brand-new store starts closed (`is_open = false`) and cannot be opened until it has at least one available menu item — the platform refuses to let an empty store go live, so customers never land on a store with nothing to order.

4.2 Vendor Dashboard

After signing in, the vendor portal shows a persistent left sidebar (Dashboard, Menu, Orders, an open/closed status pill, a My Profile section with your name, email and a Reset password link, and Sign out) with the working area on the right. The Dashboard page itself has two parts:

- Store status — an Open/Closed indicator and a toggle to close or reopen the store.
- Delivery settings — a card for editing the store's own delivery fee and an optional promo text line shown to customers.

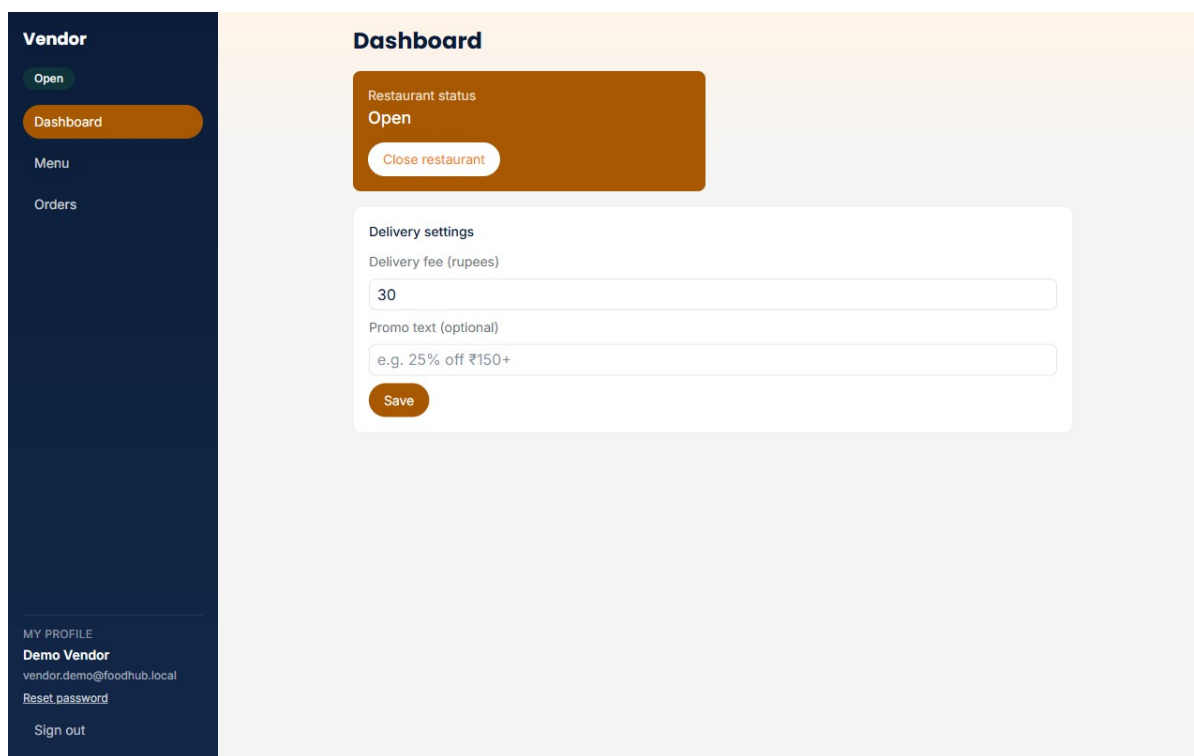


Figure 4.2 — Vendor dashboard: store status with the Open/Closed toggle, and the delivery settings card.

Step by step: opening a brand-new store

1. Sign up at /vendor/login (toggle to "New restaurant? Sign up").
2. Go to Menu and add at least one item, making sure it is marked Available.
3. Return to Dashboard and use the Open restaurant toggle — this is refused if there are zero available items.
4. Optionally set a delivery fee and promo text under Delivery settings.

4.3 Menu Management

The Menu page shows an add-item form (name, price, image URL, and the other item fields) above a table listing every existing item — its image, name, price, veg/non-veg tag, availability, and a row of actions: Edit, Mark unavailable, Delete, and Options (for item customization option groups, e.g. size or spice level, introduced in a later redesign pass).

Vendor

Open

Dashboard

Menu

Orders

Menu

Item name

Price (rupees)

Image URL (optional)

Add item

Image	Name	Price	Veg	Available	Actions			
	Paneer Butter Masala	₹220.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Veg Fried Rice	₹150.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Chicken Tikka Masala	₹240.00	—	Yes	Edit	Mark unavailable	Delete	Options
	Demo Dal Makhani	₹190.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Aloo Gobi	₹160.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Chana Masala	₹170.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Veg Kofta	₹190.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Butter Naan	₹60.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Garlic Naan	₹70.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Tandoori Roti	₹40.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Veg Pulao	₹170.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Chicken Biryani	₹260.00	—	Yes	Edit	Mark unavailable	Delete	Options
	Demo Mutton Curry	₹300.00	—	Yes	Edit	Mark unavailable	Delete	Options
	Demo Egg Curry	₹180.00	—	Yes	Edit	Mark unavailable	Delete	Options
	Demo Malai Kofta	₹220.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Rajma Chawal	₹180.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Kadhi Pakora	₹170.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Bhindi Masala	₹160.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Mixed Veg Curry	₹170.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Dal Tadka	₹160.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Palak Paneer Demo Style	₹210.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Chicken 65	₹220.00	—	Yes	Edit	Mark unavailable	Delete	Options
	Demo Veg Manchurian Demo Style	₹180.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Hakka Noodles Demo Style	₹180.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Spring Rolls Demo Style	₹140.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Chicken Momos	₹170.00	—	Yes	Edit	Mark unavailable	Delete	Options
	Demo Veg Momos	₹150.00	Veg	Yes	Edit	Mark unavailable	Delete	Options
	Demo Paneer Tikka	₹210.00	Veg	Yes	Edit	Mark unavailable	Delete	Options

Figure 4.3 — Vendor menu management page: the add-item form above the full menu table with per-row actions.

Step by step: adding a menu item

1. On the Menu page, fill in the add-item form: name, description, price, category, veg/non-veg, and an image URL.
2. Submit the form — the new item appears immediately at the top of the table below, marked Available by default.
3. Use Edit on any row to change its details later, or Mark unavailable to temporarily hide it from customers without deleting it.

Any code path that touches an item's price is written using integer-paisa arithmetic; Postgres's own numeric constraints on the products and orders tables reject invalid values, giving the platform a second line of defence even if a client-side validation were ever bypassed.

4.4 Orders (Kanban Board)

The Orders page shows every incoming order for this store as a kanban board with one column per stage: Placed, Accepted, Preparing, and Ready. A new order appears in the Placed column with Accept and Reject buttons; accepting it moves it to Accepted, from which the vendor advances it forward one stage at a time (Start preparing, then Mark ready) using a single button on the order's card. Once Ready, the order becomes visible to online delivery partners and leaves the vendor's own workflow.

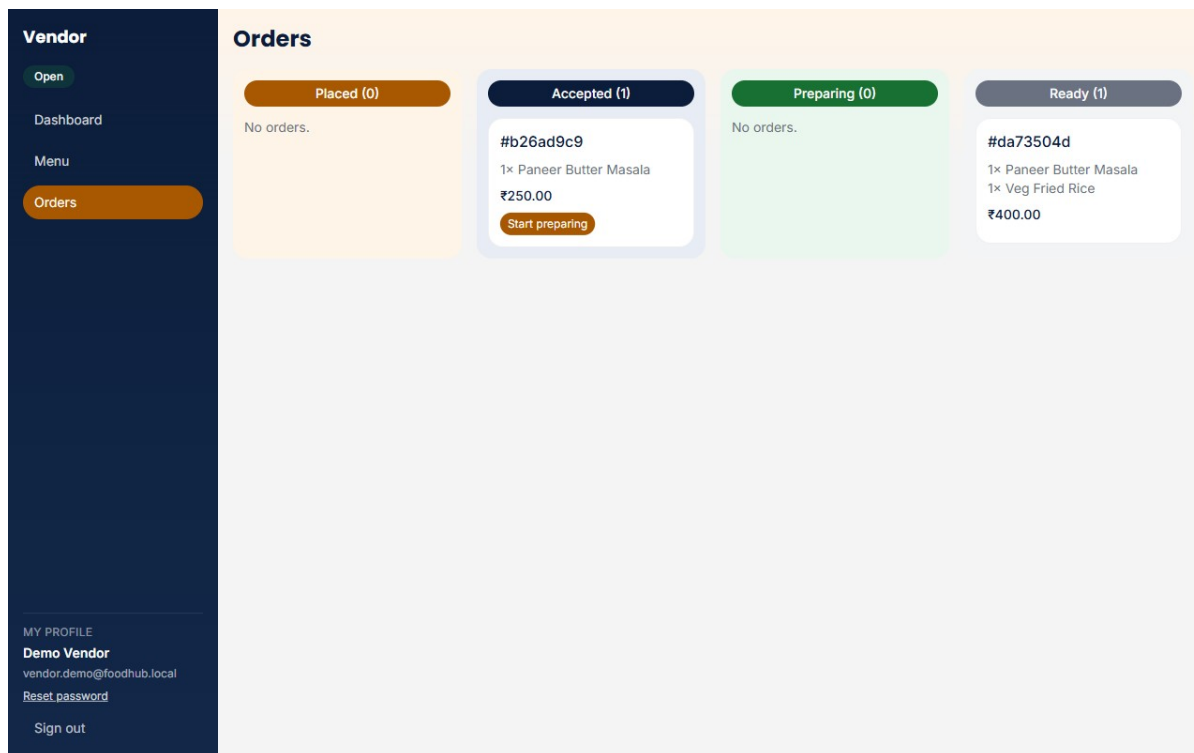


Figure 4.4 — Vendor Orders kanban board showing a real incoming order in the Placed column with Accept/Reject actions.

Step by step: fulfilling an order

1. A new order appears in the Placed column — review its items and total.
2. Select Accept to take the order, or Reject to decline it (a rejected order is cancelled outright).
3. Once accepted, select Start preparing when you begin cooking/packing.

4. Select Mark ready once the order is ready for pickup — it now appears to delivery partners as an available order.

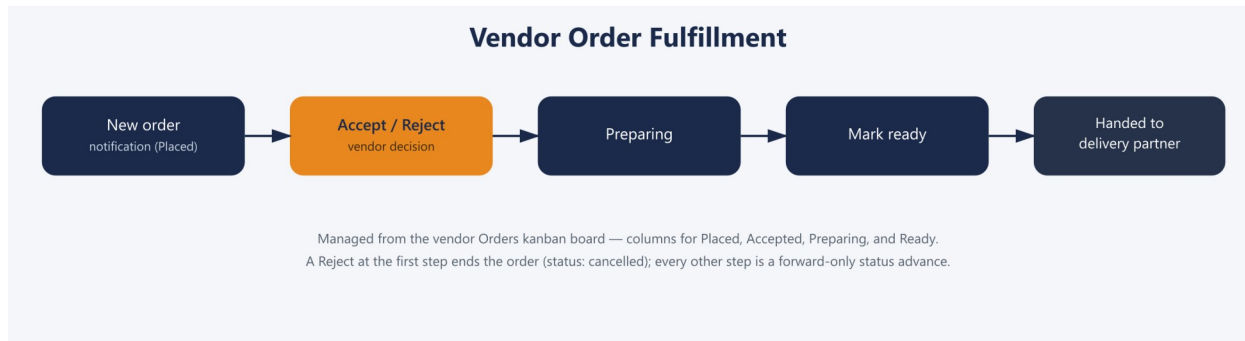


Figure 4.5 — The vendor order fulfillment flow, from a new order notification through to hand-off to a delivery partner.

Chapter 5 — Delivery Partner Module

Delivery partners sign in at their own portal to see and fulfil deliveries. Like the vendor portal, delivery partners can self-register from the login page's "New partner? Sign up" toggle, collecting email, password, full name, and vehicle type.

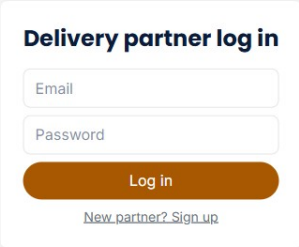
A screenshot of a web form titled "Delivery partner log in". The form is centered on a light gray background. It contains two input fields: "Email" and "Password", both with placeholder text. Below the fields is a prominent orange "Log in" button. At the bottom of the form, there is a link that reads "New partner? Sign up".

Figure 5.1 — Delivery partner sign-in page; the same screen toggles to new-partner sign-up.

5.1 Delivery Dashboard

After signing in, a persistent left sidebar shows the partner's online/offline status pill, the Dashboard link, a My Profile section (name, email and a Reset password link) and Sign out. The dashboard beside it is split into two columns:

- Available orders (left) — orders that have reached Ready status and have no delivery partner assigned yet, visible to any online partner on a first-come basis (there is deliberately no admin-assignment step for the first claim — any online partner may claim any available order).
- Your deliveries (right) — the signed-in partner's own accepted, in-progress, and completed deliveries.

An Online/Offline toggle plus two manual latitude/longitude fields stand in for live GPS tracking in this local environment; where the browser grants location permission, these fields prefill automatically from the device's own geolocation, falling back to manual entry otherwise.

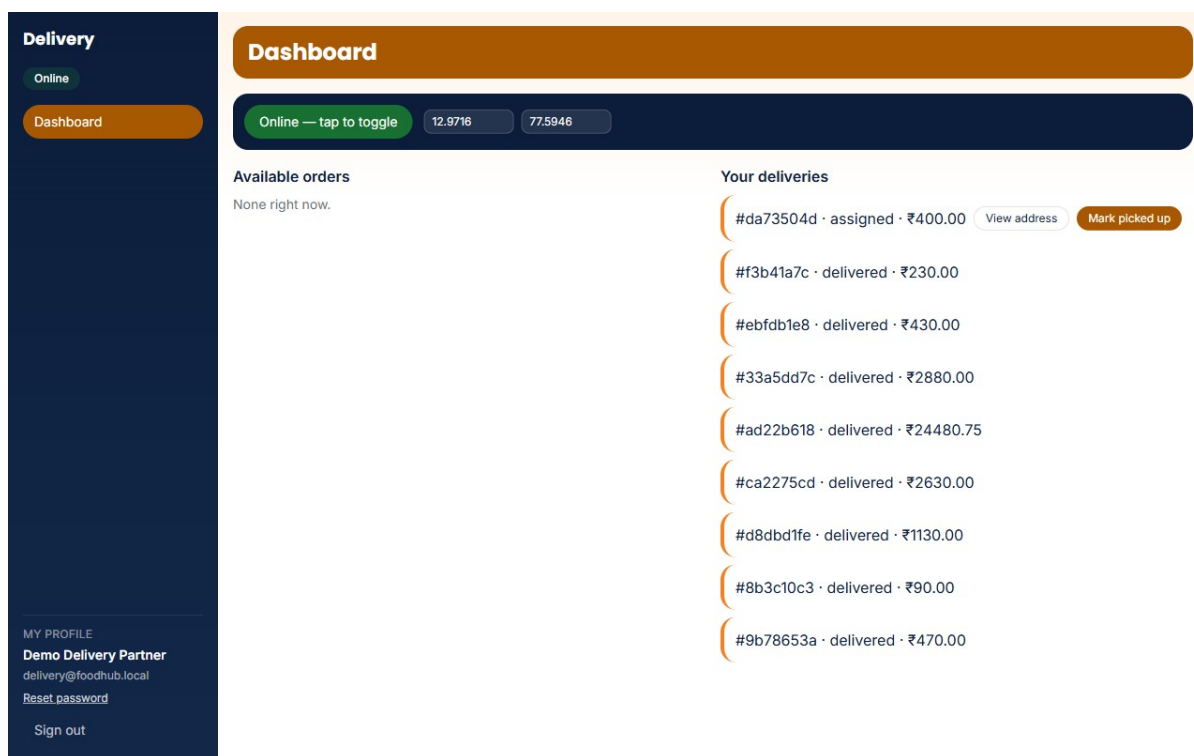


Figure 5.2 — Delivery dashboard: the online toggle and location fields, with a freshly assigned order under Your deliveries (View address / Mark picked up); the sidebar shows the My Profile section.

Step by step: fulfilling a delivery

1. Toggle Online at the top of the dashboard.
2. Select Claim on any order listed under Available orders — it moves to Your deliveries.
3. Use View address on the claimed order's card once it reaches Assigned or Picked up to see the customer's delivery address.
4. Advance the order Picked up, then Delivered, from the same card as you progress.

While online, the dashboard automatically sends a location ping every 15 seconds and polls for newly available orders every 10 seconds, so a partner does not need to manually refresh the page to see new work appear. The customer's own order-confirmation page reflects that shared location once the order reaches Assigned, and that access expires automatically once the order is Delivered.



Figure 5.3 — The delivery partner flow, from going online through to marking an order delivered.

Chapter 6 — Admin Module

The admin portal gives platform staff full oversight of the marketplace: every order, every restaurant/store, and every delivery partner, across all four portals. There is no admin self-signup — admin accounts exist only as seeded, pre-provisioned accounts, reflecting that granting admin access is an operational decision, not a public sign-up flow.

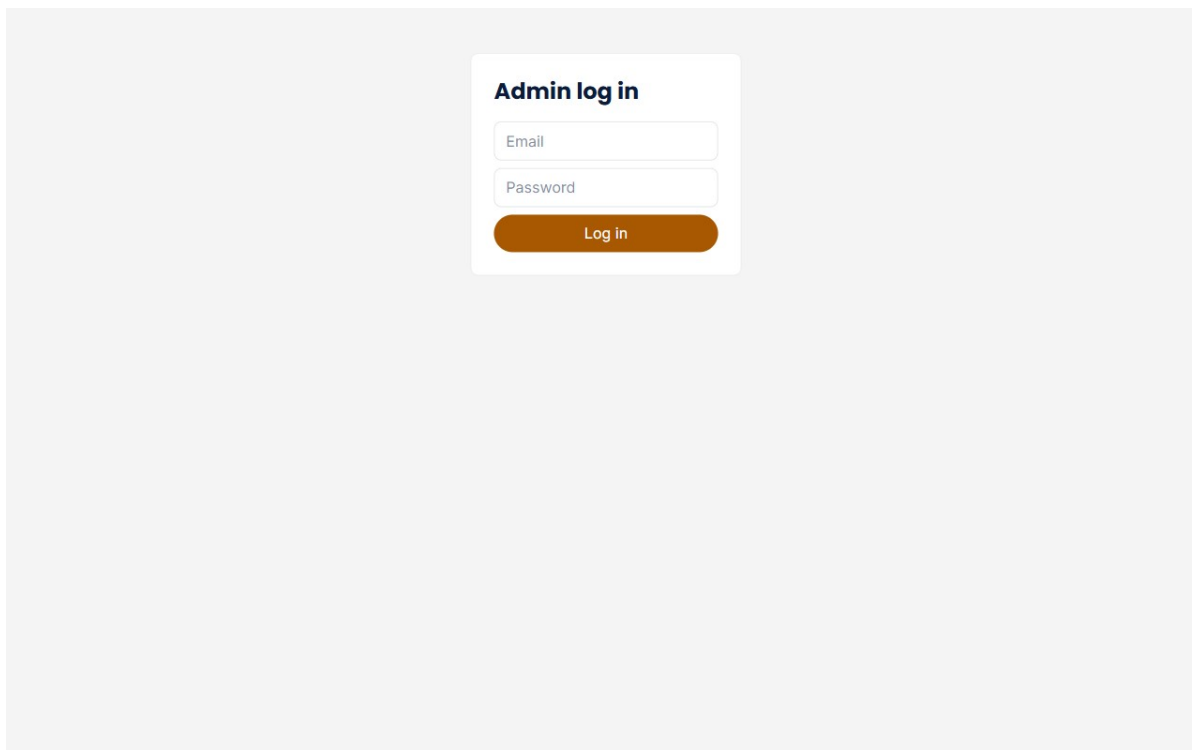


Figure 6.1 — Admin sign-in page.

6.1 Dashboard Layout

After signing in, a left sidebar shows Admin, the Dashboard link, a My Profile section (name, email and a Reset password link) and Sign out. The dashboard beside it opens with three summary tiles — Active orders, Vendors and Revenue — above a single tabbed view with three tabs: Orders, Restaurants, and Delivery partners.

6.2 Orders Tab

The Orders tab lists every order on the platform with its restaurant/store, current status, and total. For any order in the Assigned or Picked up status, a reassign dropdown (listing currently-online delivery partners) lets an admin move the order to a different partner — useful if the originally assigned partner goes offline or is unresponsive.

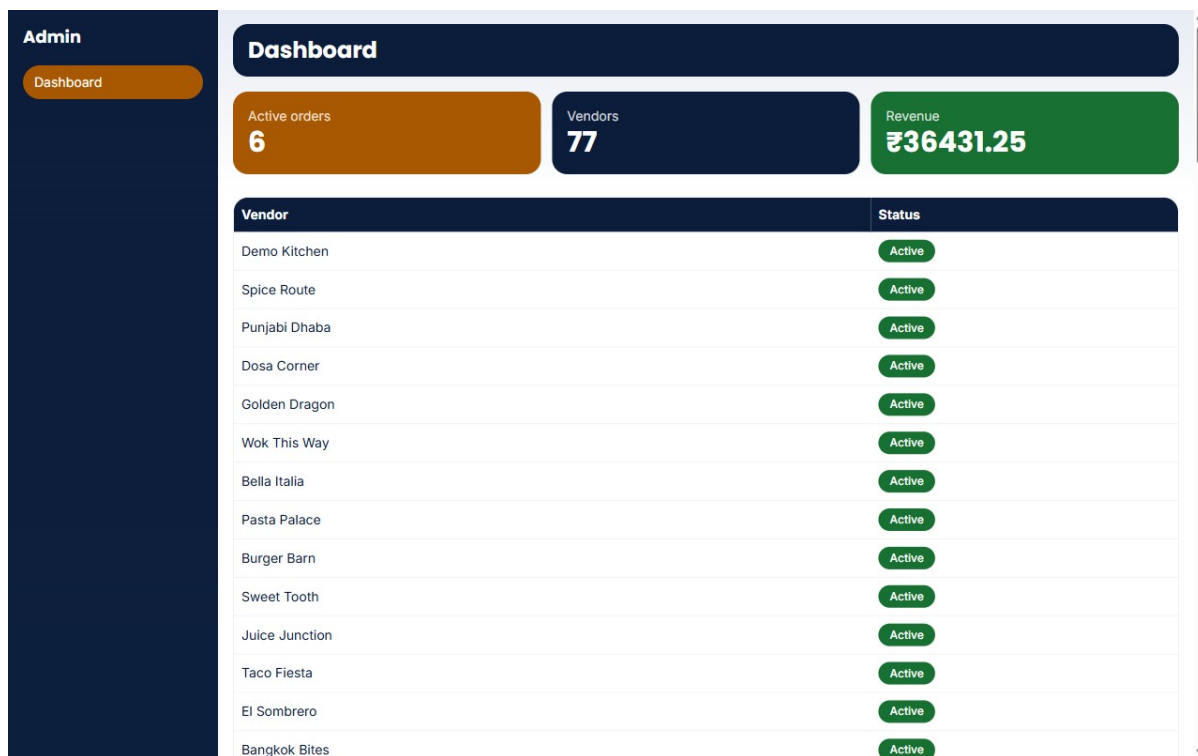


Figure 6.2 — Admin dashboard, Orders tab, listing a real order with its restaurant, status and total.

Step by step: reassigning a stuck delivery

1. Open the Orders tab and locate the order (it must be Assigned or Picked up).
2. Open its reassign dropdown, which lists every currently-online delivery partner.
3. Select a different partner — the order updates immediately and the new partner sees it under their own Your deliveries.

6.3 Restaurants Tab

The Restaurants tab lists every store on the platform — 77 in the current seed data, spanning all 11 marketplace categories — with its open/suspended status and a Suspend action. Suspending a store immediately removes it from customer browsing without deleting any of its data; unsuspending brings it back.

Happy Paws Supply	Active
Petal & Vine	Active
Bloom Street	Active
Little Nest Baby	Active
Tiny Treasures	Active
Radiant Skin Lab	Active
Shine & Glow	Active
ByteWorks	Active
Circuit City Store	Active

Orders (17)	Restaurants (77)	Delivery partners (2)
-------------	------------------	-----------------------

Name	Status	Action
Demo Kitchen	Open	Suspend
Spice Route	Open	Suspend
Punjabi Dhaba	Open	Suspend
Dosa Corner	Open	Suspend
Golden Dragon	Open	Suspend
Wok This Way	Open	Suspend
Bella Italia	Open	Suspend
Pasta Palace	Open	Suspend

Figure 6.3 — Admin dashboard, Restaurants tab, listing every seeded store with a Suspend action.

6.4 Delivery Partners Tab

The Delivery partners tab lists every delivery partner account and its current status (online/offline), giving admin staff visibility into how many partners are actively available at any given moment.

I ne Liquor Lot	Active
HealthFirst Pharmacy	Active
Nature Cure Clinic	Active
Urban Nest	Active
Trend Corner	Active
Wagging Tails	Active
Happy Paws Supply	Active
Petal & Vine	Active
Bloom Street	Active
Little Nest Baby	Active
Tiny Treasures	Active
Radiant Skin Lab	Active
Shine & Glow	Active
ByteWorks	Active
Circuit City Store	Active

Orders (17)	Restaurants (77)	Delivery partners (2)
-------------	------------------	-----------------------

Name	Status	Vehicle
Demo Delivery Partner	Online	bike
Review Rider	Offline	bike

Figure 6.4 — Admin dashboard, Delivery partners tab.

Chapter 7 — Cross-Cutting Features

7.1 Money & Pricing Precision

Every part of the platform that touches a price — menu item prices, cart subtotals, delivery fees, order totals — is computed using integer arithmetic in the smallest currency unit (paise, 1/100 of a rupee), never plain floating-point multiplication in JavaScript. This matters because floating-point money math can silently introduce rounding errors (for example, $0.1 + 0.2$ not equalling exactly 0.3 in IEEE-754 arithmetic); Postgres's own numeric CHECK constraints on the orders table (including a constraint that a stored total must exactly equal its subtotal plus delivery fee) would reject such an error outright once real, non-round decimal prices are used, rather than the kind of whole-number seed prices that could otherwise mask the bug.

7.2 Notifications & the n8n Automation Layer

A set of n8n workflows automate cross-portal side effects that would otherwise require polling: resolving a mock payment shortly after an order is placed, notifying a vendor of a new order, and similar events. n8n calls back into the Next.js application through a small family of `"/api/internal/*"` routes, authenticated with a shared secret header (X-Internal-Secret) rather than a normal user session, since these calls originate from an automation server rather than a logged-in browser. If no n8n instance is reachable, every one of these flows still completes correctly through an in-process fallback inside the relevant API route — n8n makes the demo faster, not functionally required.

7.2.1 The 5 Workflows and What Triggers Each

All 5 workflows are triggered by a Postgres Database Webhook (via the `n8n_notify()` trigger function on the orders/payments tables) firing an HTTP POST to n8n on a row INSERT or UPDATE OF status — none of the 5 is interval- or cron-based. Supabase fires the webhook on every row event for the table it is attached to; each workflow's own IF/Filter node is what narrows execution down to the specific status value(s) it cares about.

01 — Order Placed

Trigger: payments UPDATE OF status = 'success'

Calls POST `/api/internal/orders/[id]/notify-vendor`, which writes a row to `public.notifications` for the vendor. This now fires only once a payment actually succeeds, not at the moment the order is placed — earlier versions notified the vendor immediately on order placement, which could notify a vendor of an order whose payment then failed and was cancelled a moment later.

02 — Payment Mock Confirmation

Trigger: payments INSERT (status='pending')

Calls back into the app to resolve the mock payment to success/failed via `applyPaymentResult()`. The checkout route itself polls for up to ~10s and falls back to resolving the payment in-process if n8n does not answer in time, so checkout behaves identically either way.

03 — Restaurant Accepts / Status Change

Trigger: orders UPDATE OF status (accepted/preparing/ready)

Writes a placeholder notification row on every filtered transition; on accepted specifically, a second branch also emails the customer via a Gmail node (requires a connected Gmail OAuth2 credential — without one this branch fails at the Gmail node only, which does not block the order).

04 — Delivery Partner Assignment

Trigger: orders UPDATE OF status = ready

Looks up the store's lat/lng, requests nearest-first candidate delivery partners, and assigns the closest online partner automatically if any are online.

05 — Delivery Status Propagation

Trigger: orders UPDATE OF status (picked_up/delivered)

Propagates delivery-side status changes to any downstream notification/logging the workflow defines.

7.2.2 Live Verification — 2026-09-28

The local stack (self-hosted Supabase via Docker, the n8n container with N8N_INTERNAL_SECRET/APP_BASE_URL/SUPABASE_URL all set to host.docker.internal per docs/n8n-webhook-setup.md, and the Next.js dev server) was started fresh and a real order was placed end-to-end through the running customer app (test account customer@foodhub.local) to observe actual behaviour, not just configuration. Findings, checked directly against Postgres rather than assumed from n8n's execution log:

- Workflow 01 (Order Placed): confirmed firing. The order was placed at 00:13:42 UTC and a matching row appeared in public.notifications at 00:13:44.98 — about 2.6 seconds later, consistent with a live webhook round trip rather than the app's own code creating that row synchronously.
- Workflow 02 (Payment Mock Confirmation): confirmed firing. The order's payment moved from pending to success at 00:13:46.78 — about 4.4 seconds after checkout, well inside the ~10-second in-process fallback window documented in section 7.2 and Appendix A, so this resolution was n8n's, not the fallback's.
- Workflow 03 (Restaurant Accepts): the seeded vendor account (daily-basket@foodhub.local) advanced the same order through accepted → preparing → ready via /vendor/orders in this session, which is the real trigger for workflow 03's filtered UPDATE OF status. The Gmail-send branch was not separately confirmed (no Gmail credential is configured in this local n8n instance, matching the known gap in section 4 of docs/n8n-webhook-setup.md), so this document does not claim that branch fired — only that the accepted/preparing/ready transitions themselves, which are what the workflow listens for, occurred live.
- Workflow 04 (Delivery Partner Assignment): NOT observed to auto-assign in this test. With the order at status=ready and a delivery partner already online (delivery@foodhub.local), the order appeared under that partner's "Available orders" list for manual self-claim rather than being auto-assigned by the workflow — orders.delivery_partner_id remained null until the partner claimed it by hand through /delivery/dashboard. This manual is deliberately not claiming workflow 04 fired successfully; it may not be active in this local n8n instance, or its assignment call may not have succeeded silently. Either way, the self-claim path (section 5.2) worked correctly as the primary, always-available assignment mechanism, exactly as designed — workflow 04 is additive automation on top of it, not a replacement for it.
- Workflow 05 (Delivery Status Propagation): not separately exercised in this pass (the test order was left at the claimed/assigned stage); see docs/n8n-webhook-setup.md section 5 for its documented test procedure.

Net conclusion: this local instance's checkout-critical automation (workflows 01 and 02) is live, active, and measurably faster than its own documented fallback — not merely configured on paper. Workflow 04's auto-assignment did not visibly act in this run, so treat delivery self-claim as the mechanism to rely on and workflow 04 as an unconfirmed bonus in this environment rather than a guaranteed path.

7.3 Payment Methods

Three payment methods are available at checkout, described in full in section 3.4: Mock Card and Mock UPI (each with method-specific required fields, and each resolving to success or failure with roughly an 80% success rate to simulate a real gateway), and Cash on Delivery (no extra fields, always succeeds). All three are simulated — no real payment processor is integrated, appropriate for a local development and demonstration environment.

7.4 Categories & the Marketplace Taxonomy

The platform's category system (11 categories: Restaurants plus the 10 non-restaurant categories listed throughout this manual) and cuisine taxonomy (Indian, Chinese, Italian, Mexican, and more) are both enforced as fixed, predefined lists at the database level, not just in the UI — a database trigger rejects any cuisine tag or category value outside the recognised set. This keeps every filter, chip, and carousel on the customer home page working against a known, closed set of values rather than free-text tags that could drift or typo over time.

Appendix A — Technical Architecture & Stack

A.1 Technology Stack

Layer	Technology
Frontend framework	Next.js (App Router, TypeScript) + Tailwind CSS
Backend	Next.js API routes (server-side), running in the same codebase as the frontend
Database	Postgres, via self-hosted Supabase (Docker) — no cloud/hosted Supabase project
Auth	Supabase Auth (email/password), one role column driving portal redirect
Storage	Supabase Storage (self-hosted)
Realtime / polling	Supabase Realtime plus client-side polling for order status and available-order lists
Automation	n8n (self-hosted, Docker), calling back via internal API routes
Row security	Postgres Row-Level Security (RLS) policies, scoped per role and per table

A.2 Why Self-Hosted, Local-Only

A deliberate project decision (documented from the platform's earliest design phase) keeps every backing service — Postgres, Auth, Storage, and the n8n automation layer — running in local Docker containers rather than any hosted cloud service. This keeps the running cost at zero and keeps the entire stack reproducible on a single developer machine, at the cost of features that would need a real cloud deployment (e.g. a public webhook endpoint reachable from outside the local machine).

A.3 Security Model

- The Supabase "service role" key — which bypasses all row-level security — is only ever imported in server-side files explicitly guarded by the server-only package, so it can never end up in a browser-shipped JavaScript bundle.
- Any API route that creates data on a customer's behalf derives that customer's identity from a verified session token (checked server-side against Supabase Auth), never from a client-supplied ID in the request body.
- Checkout runs through a single Postgres function (a "security definer" RPC) so that an order's four related inserts — the order row, its line items, and its payment record — happen atomically; that function is granted only to the service role, so a customer's own browser session cannot call it directly and bypass the checkout route's own validation.
- Row-level security write policies exist only on tables a browser client is genuinely meant to write to directly; every other table is written exclusively through service-role-backed API routes, closing off a class of bug where a leftover "just in case" RLS write policy becomes a live bypass of an API route's own validation.

A.4 Architecture Diagram

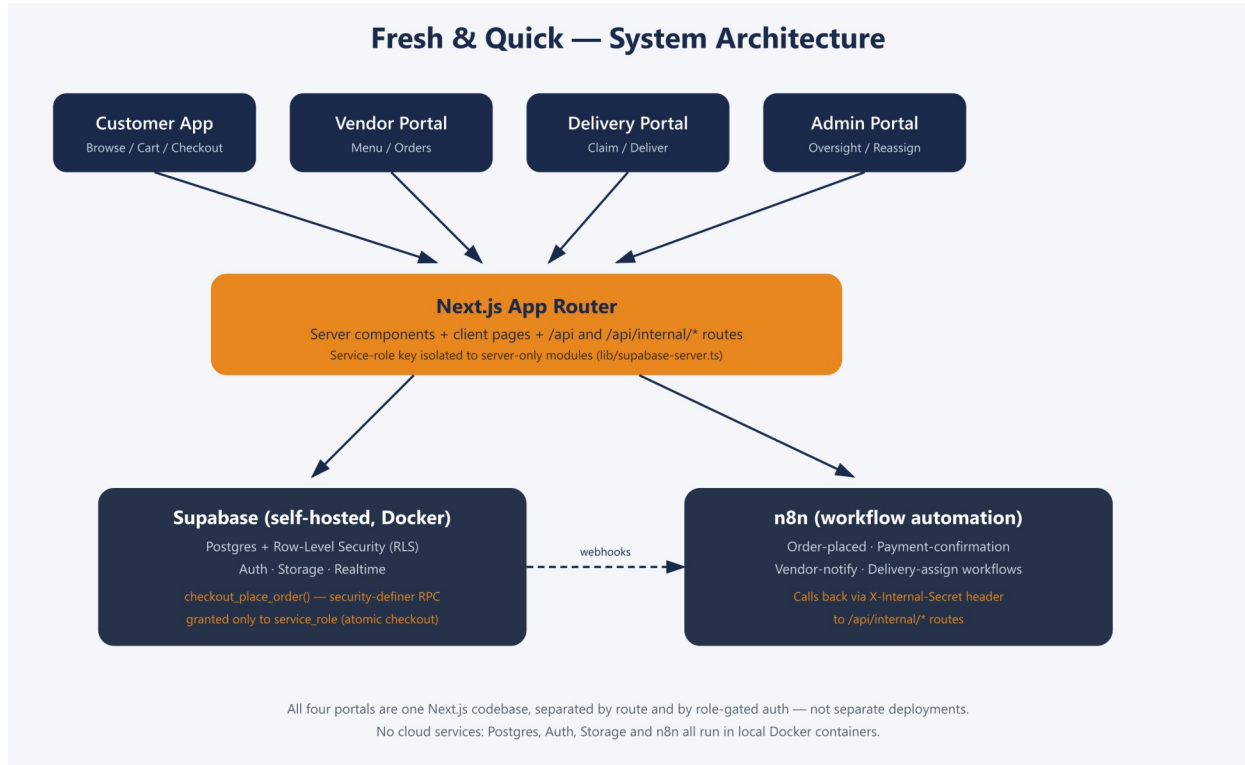


Figure A.1 — System architecture (repeated from Chapter 1 for reference).

Appendix B — Database Schema Reference

The tables below summarise the platform's core Postgres schema. Every table has Row-Level Security enabled; most read access is public or authenticated-only, while all sensitive writes route through service-role-backed API routes rather than direct client writes.

B.1 users

Extends Supabase's own `auth.users` table with an application profile: a role (customer, vendor, delivery, or admin — this single column drives which portal a signed-in user is redirected to), full name, phone, and avatar URL.

B.2 addresses

A saved delivery address for a user: a label, a structured street address (line1, an optional line2, city, state, and zipcode), and a latitude/longitude pair captured from the manual location-entry stub.

B.3 stores

Represents any store on the platform, not only food restaurants — the same table backs every one of the 11 marketplace categories. Key columns: owner (the vendor's user id), name, an array of cuisine/category tags, a location (address reference plus lat/long), an open/closed flag, average prep time in minutes, a rating, and a banner image URL.

B.4 products

An item in a store's catalog: name, description, price (checked non-negative at the database level), category, a veg/non-veg flag, an availability flag, and an image URL. Belongs to exactly one restaurant/store row.

B.5 orders

The central transactional record: customer, restaurant/store, an optional assigned delivery partner, a delivery address, and a status that moves forward through a fixed chain — placed, accepted, preparing, ready, assigned, picked_up, delivered, or cancelled at any point. Subtotal, delivery fee, and total are each checked non-negative, and a database constraint requires total to exactly equal subtotal plus delivery fee, closing off an entire class of pricing bug at the data layer.

B.6 order_items

One row per line item within an order: which menu item, what quantity (must be positive), and the unit price actually charged at the time of the order (stored independently of the menu item's current price, so a later price change never rewrites the price on a historical order).

B.7 delivery_partners

One row per delivery-partner user: online/offline flag, current latitude/longitude and the timestamp of the last location ping, and vehicle type. This is the table the delivery dashboard's Online toggle and location-ping feature read from and write to.

B.8 payments

One row per order's payment attempt: method (mock_card, mock_upi, or mock_cod), status (pending, success, or failed), amount, a mock reference string standing in for a real payment gateway's transaction id, and a paid-at timestamp.

B.9 reviews

An optional post-delivery review: which order, which customer, which restaurant/store, a 1–5 star rating, and an optional comment. This table exists in the schema from the platform's first migration onward but does not yet have a customer-facing UI for leaving a review.

B.10 Schema Summary Table

Table	Purpose	Key columns
users	App profile layered on Supabase Auth	role, full_name, phone
addresses	Saved delivery addresses	line1, line2, city, state, pincode, lat, lng, is_default
stores	Any store (all 11 categories)	name, cuisine_tags[], is_open, rating
products	Catalog items for a store	name, price, category, is_available
orders	Central order record	status, subtotal, delivery_fee, total
order_items	Order line items	menu_item_id, quantity, unit_price
delivery_partners	Partner online status & location	is_online, current_lat, current_lng
payments	Payment attempts per order	method, status, amount
reviews	Post-delivery reviews	rating, comment

Appendix C — Seeded Test Accounts

Re-running `npx supabase db reset` restores exactly these 80 accounts with exactly these passwords every time, making the environment fully reproducible for testing or demonstration. All passwords below are dummy, development-only credentials, seeded locally, and are never used outside a local Supabase instance.

C.1 Core Accounts (one per role)

Role	Login page	Email	Password
Admin	/admin/login	admin@foodhub.local	admin-demo-password
Customer	/customer/login	customer@foodhub.local	demo1234
Delivery Partner	/delivery/login	delivery@foodhub.local	demo1234
Vendor (Demo Kitchen)	/vendor/login	vendor.demo@foodhub.local	demo1234

C.2 Restaurant Vendor Accounts

All 16 restaurants share the password `demo1234`; sign in at `/vendor/login` with any of these to manage that restaurant's menu and orders.

Restaurant	Email	Cuisine
Spice Route	spice-route@foodhub.local	indian
Punjabi Dhaba	punjabi-dhaba@foodhub.local	north_indian
Dosa Corner	dosa-corner@foodhub.local	south_indian
Golden Dragon	golden-dragon@foodhub.local	chinese
Wok This Way	wok-this-way@foodhub.local	chinese
Bella Italia	bella-italia@foodhub.local	italian
Pasta Palace	pasta-palace@foodhub.local	italian
Burger Barn	burger-barn@foodhub.local	fast_food
Sweet Tooth	sweet-tooth@foodhub.local	desserts
Juice Junction	juice-junction@foodhub.local	beverages
Taco Fiesta	taco-fiesta@foodhub.local	mexican
El Sombrero	el-sombrero@foodhub.local	mexican
Bangkok Bites	bangkok-bites@foodhub.local	thai
The Bread Basket	the-bread-basket@foodhub.local	bakery
Green Bowl	green-bowl@foodhub.local	healthy
Fresh Fit	fresh-fit@foodhub.local	healthy

C.3 Non-Restaurant Marketplace Vendor Accounts

Six stores per category (all sharing the password demo1234), grouped below by category. Sign in at /vendor/login.

C.3.1 Grocery

Store	Email	Category
Daily Basket	daily-basket@foodhub.local	Grocery
Fresh Mart	fresh-mart@foodhub.local	Grocery
Green Grocer	green-grocer@foodhub.local	Grocery
Harvest Greens	harvest-greens@foodhub.local	Grocery
Metro Grocery	metro-grocery@foodhub.local	Grocery
Pantry Post	pantry-post@foodhub.local	Grocery

C.3.2 Convenience

Store	Email	Category
24/7 Shop	247-shop@foodhub.local	Convenience
24Seven Store	24seven-store@foodhub.local	Convenience
Corner Store	corner-store@foodhub.local	Convenience
QuickBite Mart	quickbite-mart@foodhub.local	Convenience
Metro Mart Express	metro-mart-express@foodhub.local	Convenience
QuickStop	quickstop@foodhub.local	Convenience

C.3.3 Alcohol

Store	Email	Category
Barrel & Bottle	barrel-and-bottle@foodhub.local	Alcohol
Craft Beer Co.	craft-beer-co@foodhub.local	Alcohol
Spirits & More	spirits-and-more@foodhub.local	Alcohol
The Bottle Shop	the-bottle-shop@foodhub.local	Alcohol
The Liquor Loft	the-liquor-loft@foodhub.local	Alcohol
The Wine Cellar	wine-cellar@foodhub.local	Alcohol

C.3.4 Health

Store	Email	Category
Care & Cure	care-and-cure@foodhub.local	Health
HealthFirst Pharmacy	healthfirst-	Health

Store	Email	Category
	pharmacy@foodhub.local	
MedPlus Pharmacy	medplus-pharmacy@foodhub.local	Health
Nature Cure Clinic	nature-cure-clinic@foodhub.local	Health
Vitamin Shop	vitamin-shop@foodhub.local	Health
WellnessRx	wellness-rx@foodhub.local	Health

C.3.5 Retail

Store	Email	Category
Fashion Hub	fashion-hub@foodhub.local	Retail
Home Essentials	home-essentials@foodhub.local	Retail
Style Bazaar	style-bazaar@foodhub.local	Retail
Trend Corner	trend-corner@foodhub.local	Retail
Urban Living	urban-living@foodhub.local	Retail
Urban Nest	urban-nest@foodhub.local	Retail

C.3.6 Pet

Store	Email	Category
Furry Friends Mart	furry-friends-mart@foodhub.local	Pet
Happy Paws Supply	happy-paws-supply@foodhub.local	Pet
Paws & Claws	paws-claws@foodhub.local	Pet
The Pet Corner	pet-corner@foodhub.local	Pet
Wagging Tails	wagging-tails@foodhub.local	Pet
Whiskers & Wags	whiskers-wags@foodhub.local	Pet

C.3.7 Flowers

Store	Email	Category
Bloom & Blossom	bloom-blossom@foodhub.local	Flowers
Bloom Street	bloom-street@foodhub.local	Flowers
Fresh Petals Co.	fresh-petals-co@foodhub.local	Flowers
Petal Bloom	petal-bloom@foodhub.local	Flowers
Petal & Vine	petal-and-vine@foodhub.local	Flowers
The Flower Basket	flower-basket@foodhub.local	Flowers

C.3.8 Baby

Store	Email	Category
BabyCare Hub	babycare-hub@foodhub.local	Baby
Cuddle & Co.	cuddle-co@foodhub.local	Baby
Little Nest Baby	little-nest-baby@foodhub.local	Baby
Little Steps	little-steps@foodhub.local	Baby
Tiny Tots Store	tiny-tots-store@foodhub.local	Baby
Tiny Treasures	tiny-treasures@foodhub.local	Baby

C.3.9 Personal Care

Store	Email	Category
Beauty Bar	beauty-bar@foodhub.local	Personal Care
Glow Beauty	glow-beauty@foodhub.local	Personal Care
Grooming Co.	grooming-co@foodhub.local	Personal Care
Pure Skin	pure-skin@foodhub.local	Personal Care
Radiant Skin Lab	radiant-skin-lab@foodhub.local	Personal Care
Shine & Glow	shine-and-glow@foodhub.local	Personal Care

C.3.10 Electronics

Store	Email	Category
ByteWorks	byteworks@foodhub.local	Electronics
CircuitPoint	circuitpoint@foodhub.local	Electronics
Digital Hub	digital-hub@foodhub.local	Electronics
Gadget World	gadget-world@foodhub.local	Electronics
Circuit City Store	circuit-city-store@foodhub.local	Electronics
TechZone	techzone@foodhub.local	Electronics

Appendix D — Glossary of Terms

Term	Meaning
Portal	One of the four role-specific areas of the site: Customer, Vendor, Delivery, Admin.
Vendor	A seller account managing exactly one store — a restaurant or any other marketplace category store.
Store / Restaurant row	The database and UI term for any seller's catalog, whether food or non-food.
RLS (Row-Level Security)	A Postgres feature restricting which rows a given database role may read or write, enforced at the database itself rather than only in application code.
Service role	A privileged Supabase key that bypasses RLS, used only in server-side API routes, never exposed to the browser.
RPC (Remote Procedure Call)	A named Postgres function called directly (here, the checkout function that atomically creates an order).
n8n	A self-hosted workflow-automation tool used here to resolve payments and send cross-portal notifications after key events.
Mock payment	A simulated (non-real) payment method — Mock Card, Mock UPI, or Cash on Delivery — used since no real payment gateway is integrated.
Available order	An order that has reached Ready status and has no delivery partner assigned yet, visible to online delivery partners to claim.
Seed data	The fixed set of demo accounts, stores, and menu items loaded by npx supabase db reset, used for local development and testing.
Paisa	1/100 of an Indian Rupee; all money in the platform is computed and stored in whole paise to avoid floating-point rounding errors.